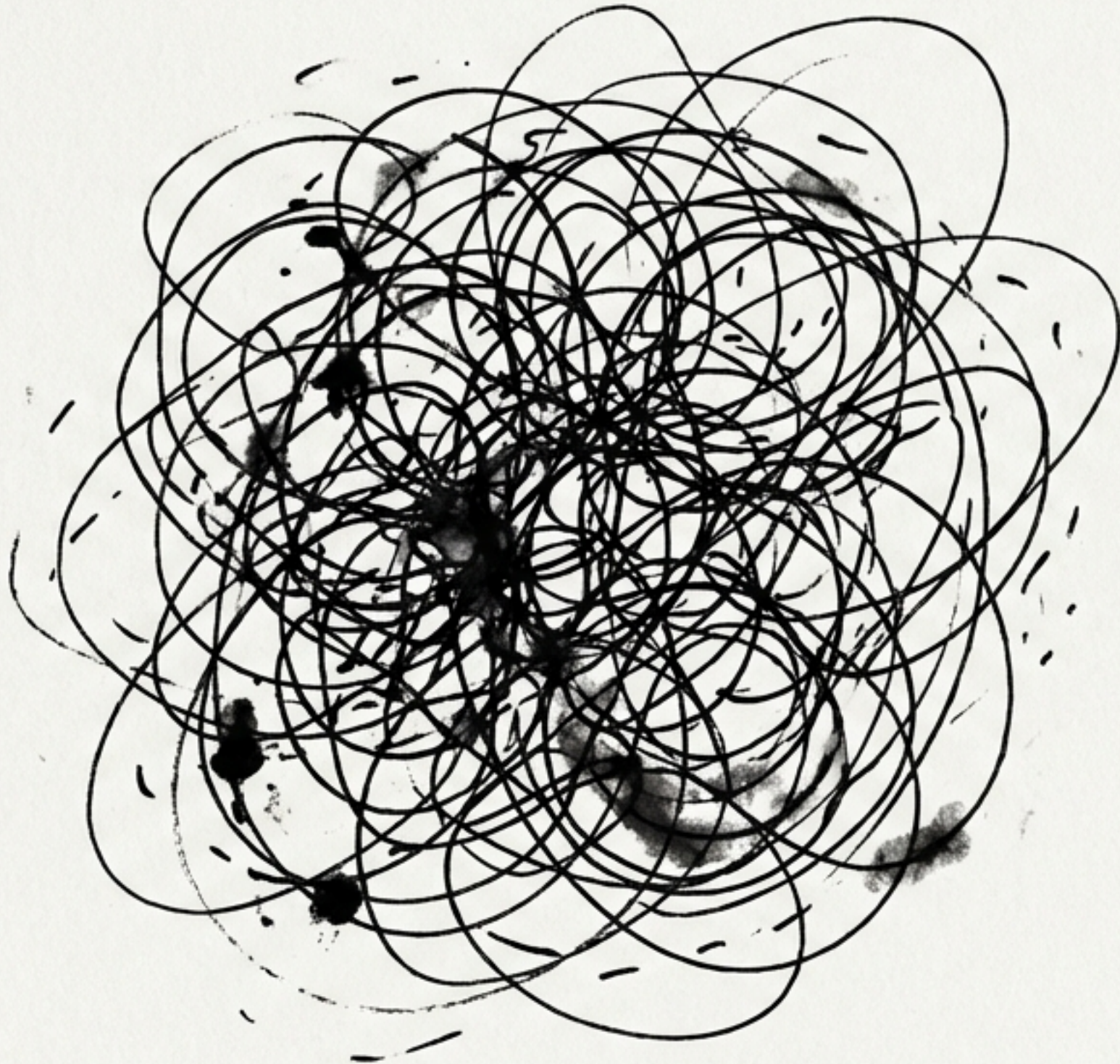


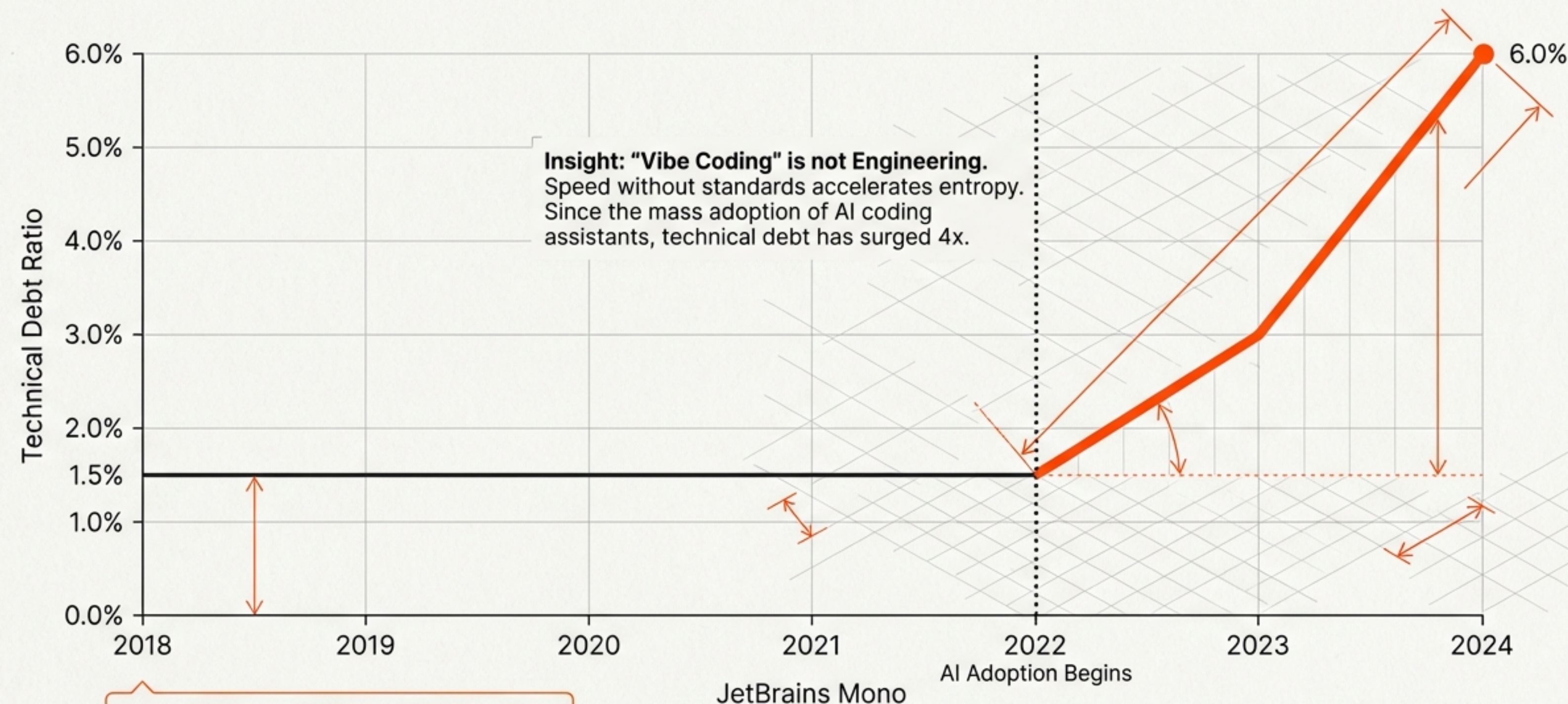


SPEC-DRIVEN DEVELOPMENT

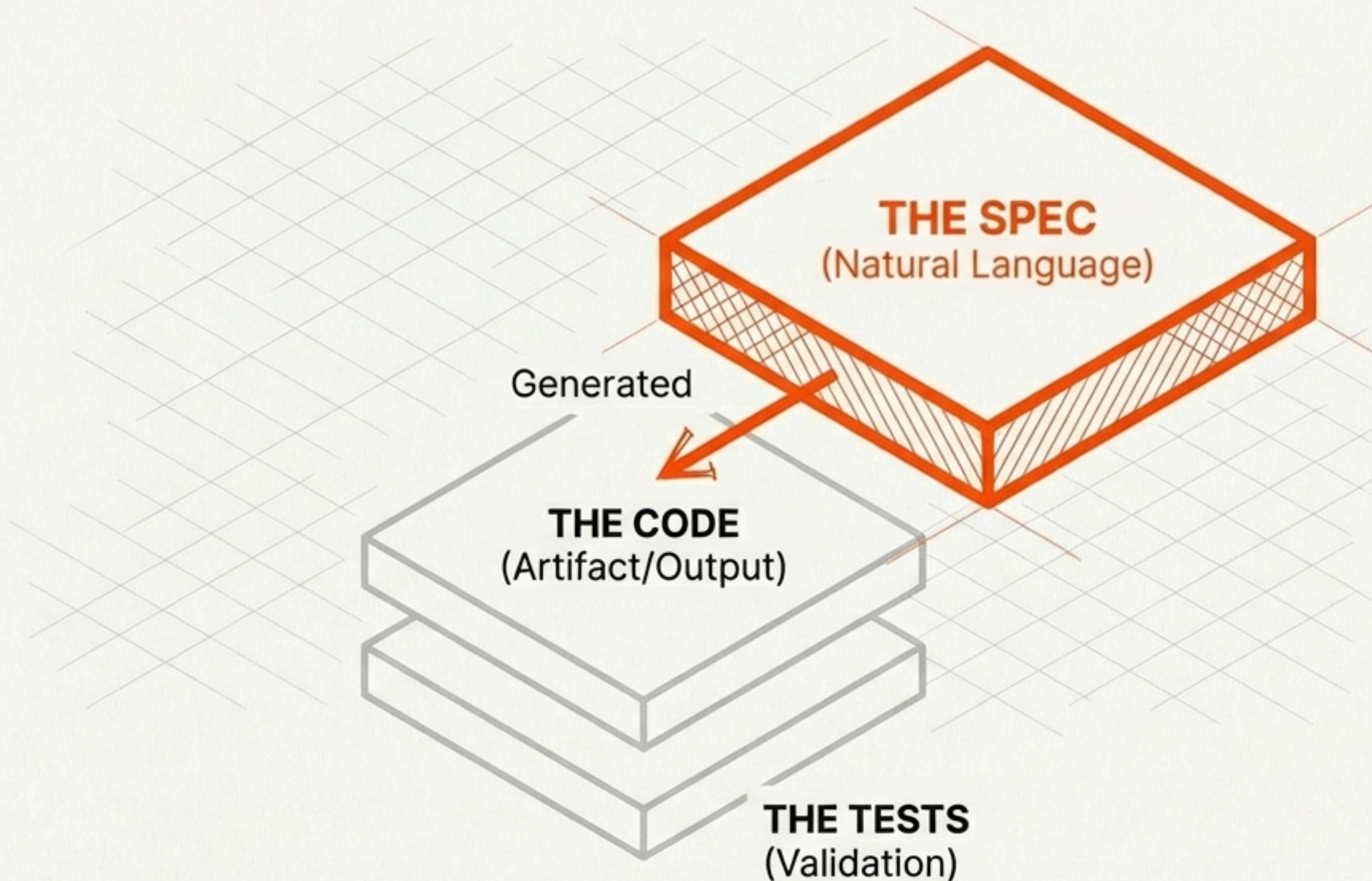
Turning 'Vibe Coding' into Rigorous
Software Engineering.

Methodology: DevPills Technical
Walkthrough & Open Spec Framework

THE HIDDEN COST OF AI WITHOUT PROCESS



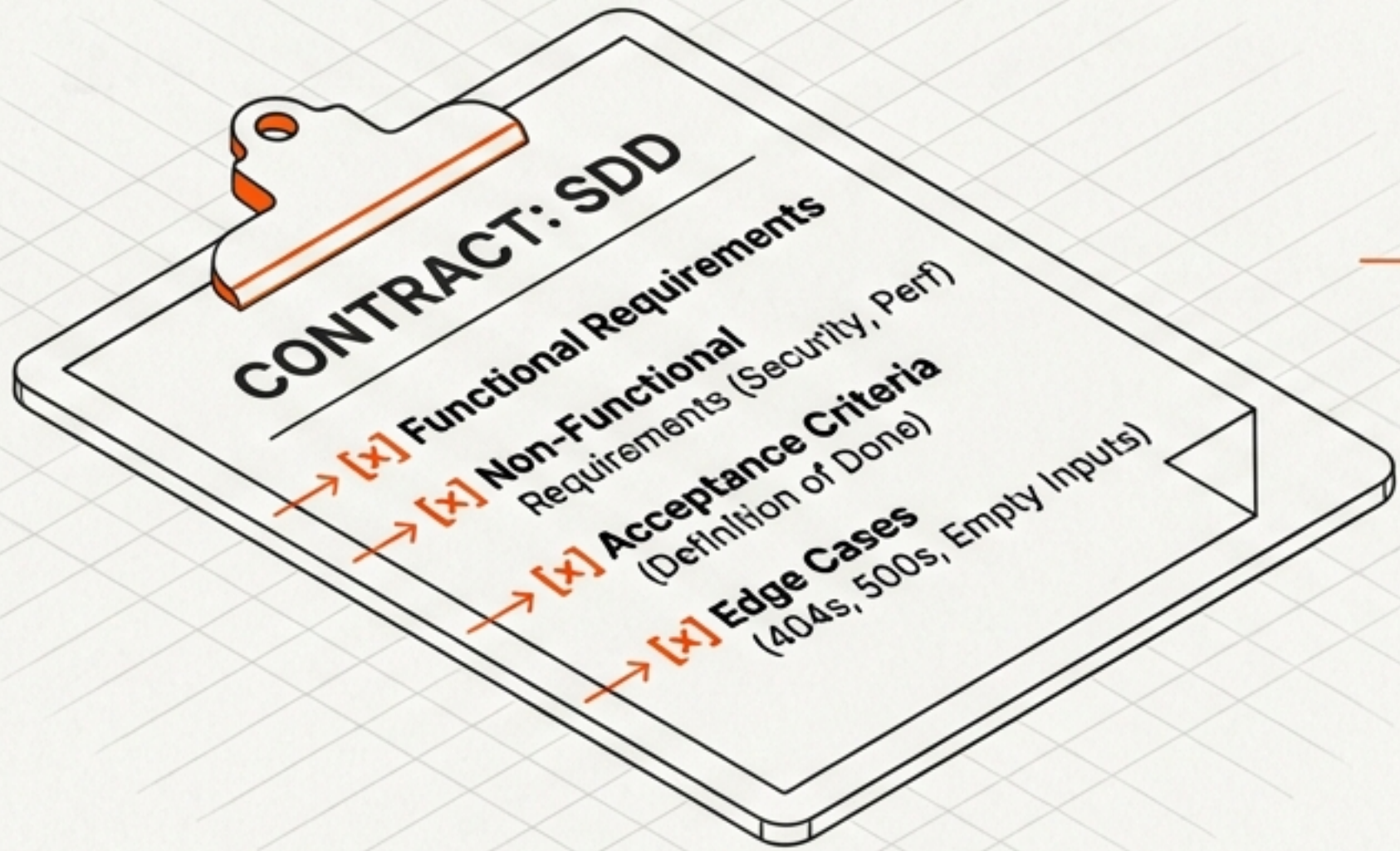
THE NEW SOURCE OF TRUTH



Core Shift: Code is no longer the documentation. The Spec is the documentation. Development is guided by formal specifications that serve as the single source of truth.

Why? Instant code generation allows time to be reinvested in defining the 'What' rather than typing the 'How'.

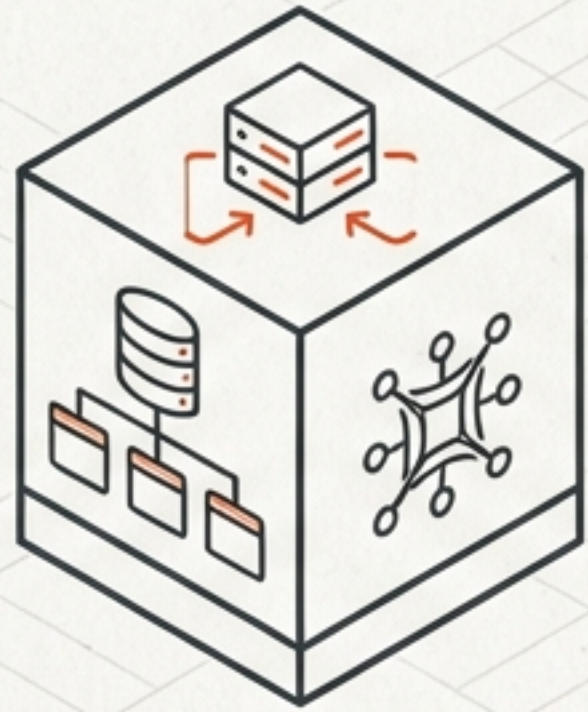
THE ENGINEERING CONTRACT



Eliminating Hallucinations.

SDD creates a binding agreement between the Human Architect and the AI Executor. Without validatable constraints, the AI guesses—and usually guesses wrong.

THE SECRET INGREDIENT IS CONTEXT



TECHNICAL SPECS

Stack, Schema, Patterns



WORKFLOW RULES

Testing, PR Reviews, Naming



USER STORY

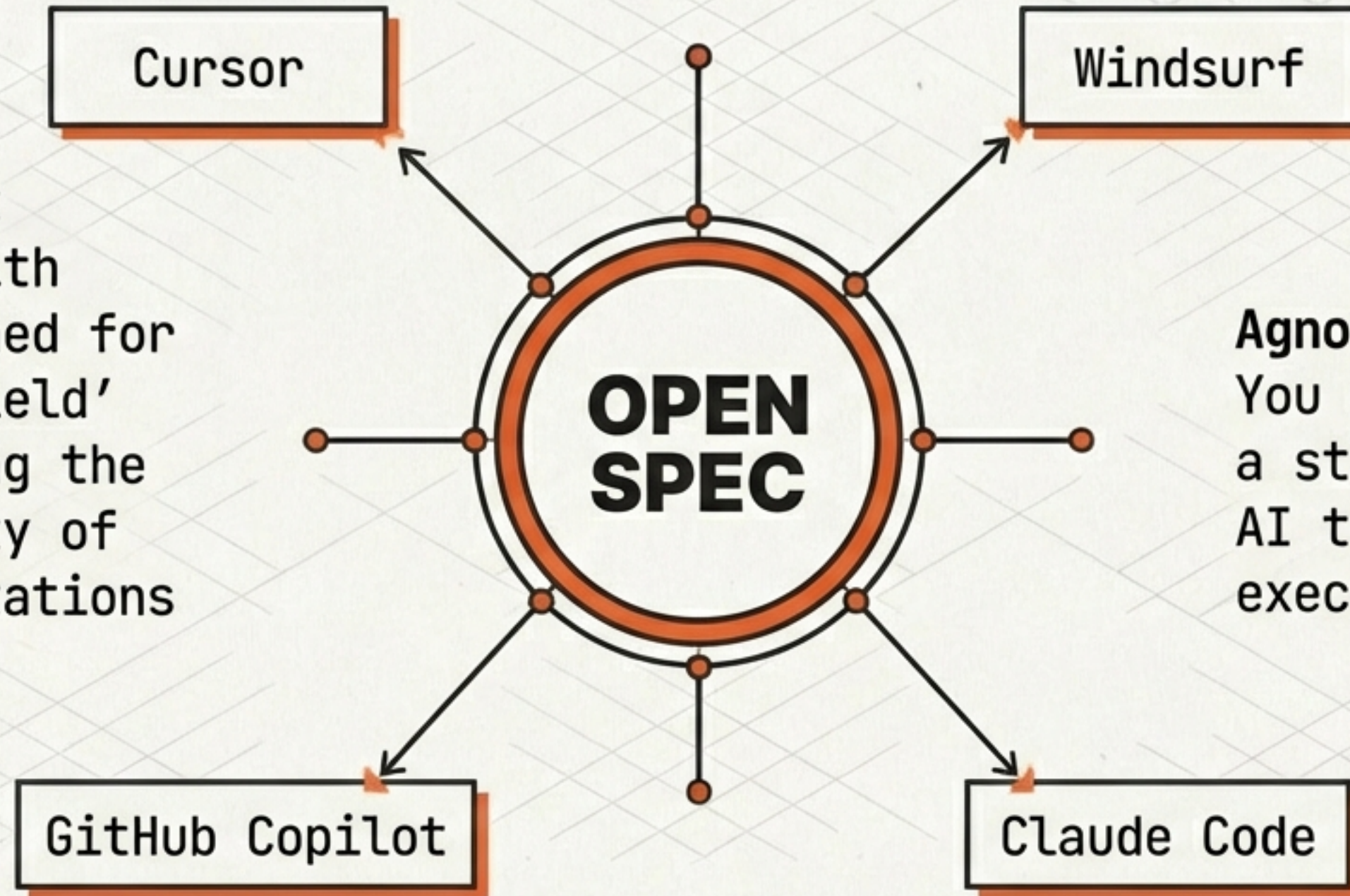
The specific task



If you don't document your context, the AI lacks the 'institutional knowledge' of a senior engineer. This context must live in the repo, not in Slack.

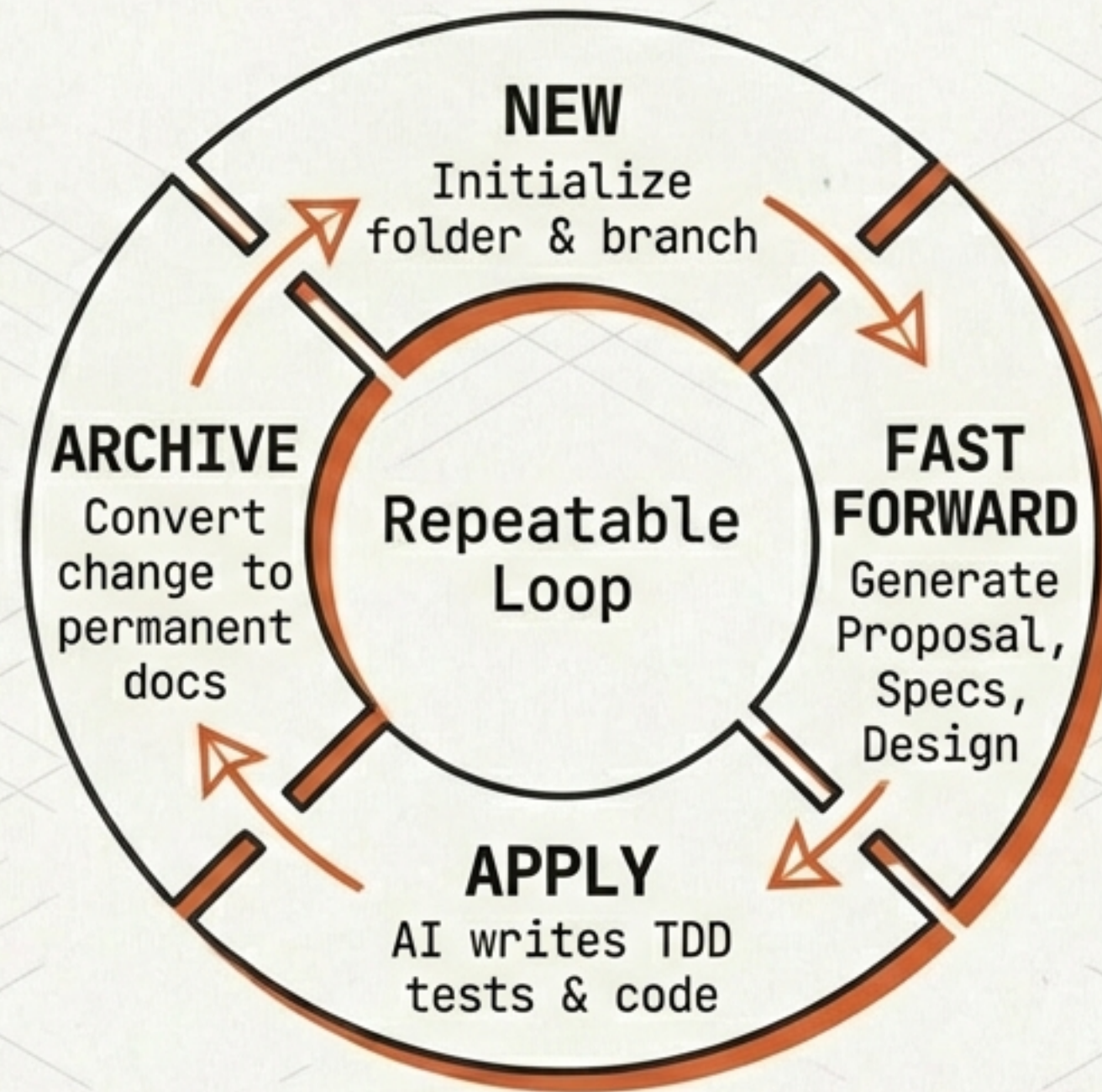
THE TOOLING ECOSYSTEM

Why Open Spec? It balances power with usability. Designed for existing 'brownfield' projects, avoiding the extreme complexity of BMAT or the limitations of Spec Kit.



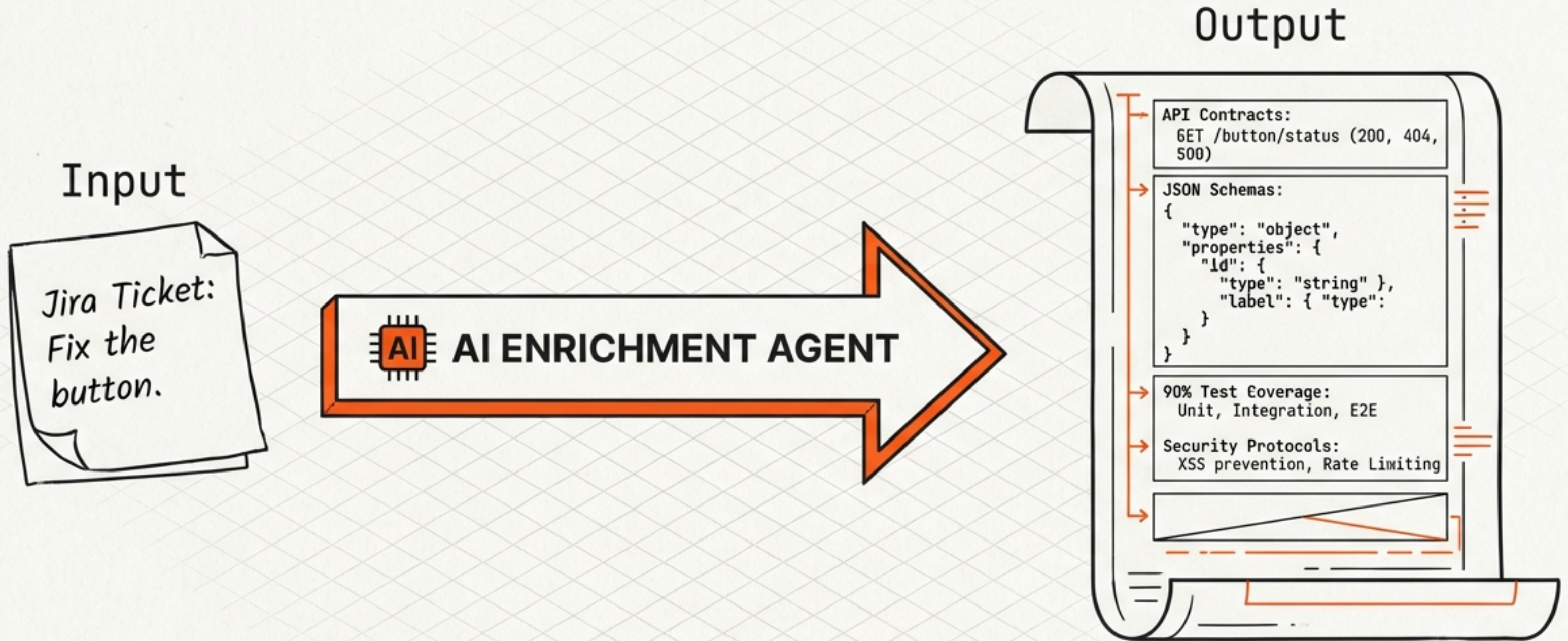
Agnostic Framework:
You define the specs in a standard format; the AI tool of your choice executes them.

THE SDD LIFECYCLE



Four terminal commands create a standardized workflow that works the same for a Junior or Senior developer.

STEP 1: ENRICH THE INTENT



Garbage In, Garbage Out. The 'Enrich' script expands a lazy human intent into a massive, rigorous technical blueprint before coding begins.

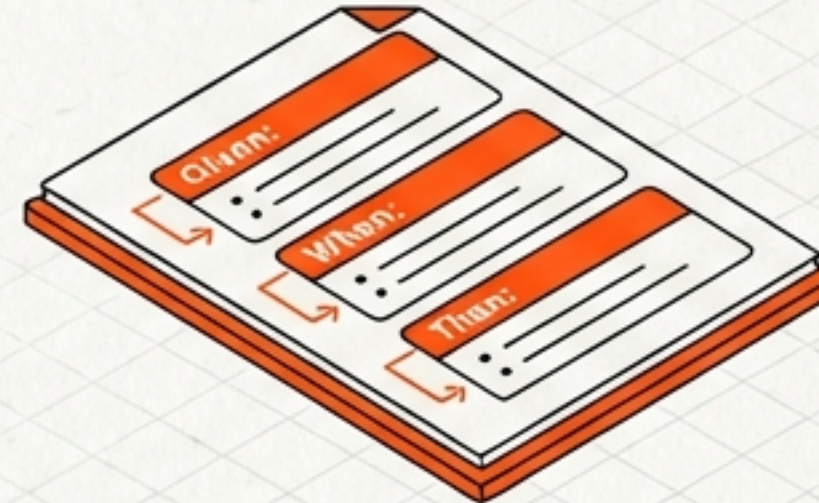
STEP 2: GENERATING THE ARTIFACTS

The "Fast Forward" command generates the plan, not just the code.



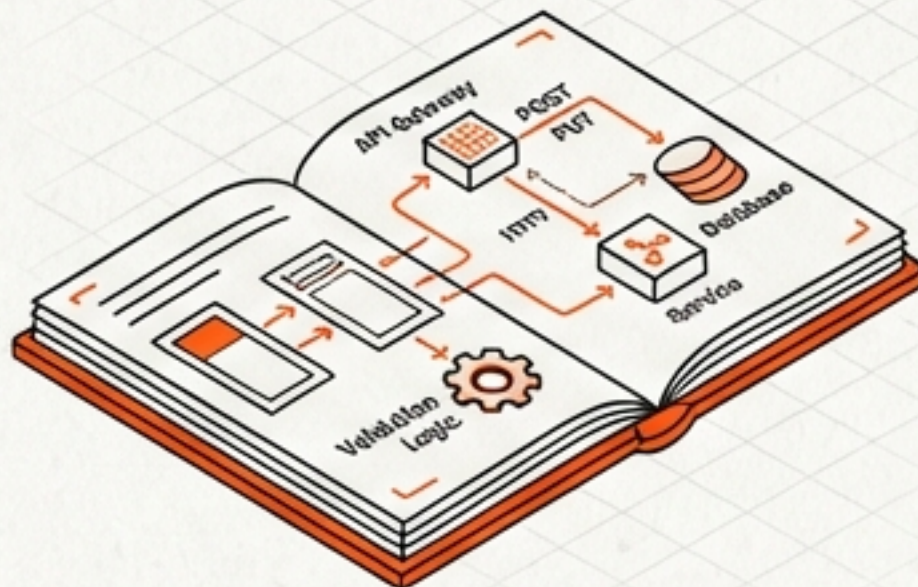
PROPOSAL

The "Why" and high-level approach.



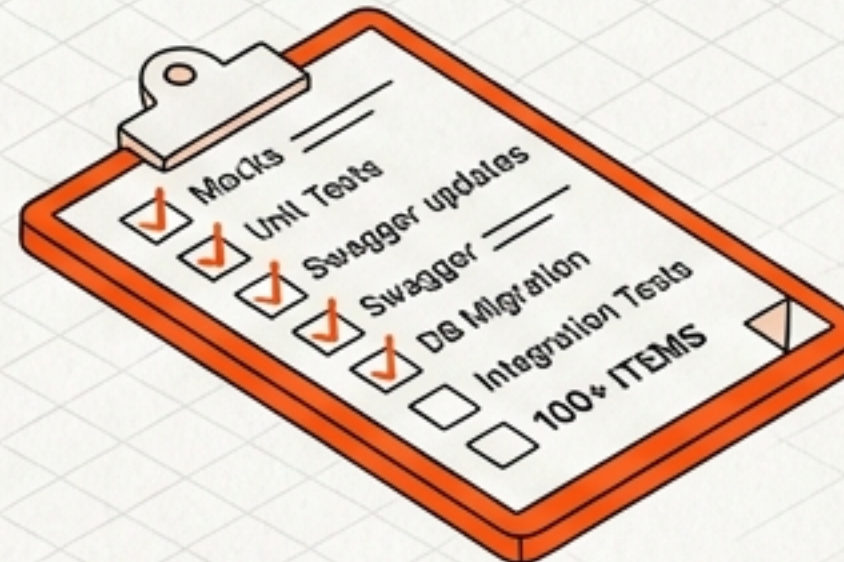
SPECS

Requirements in BDD/Gherkin format (Given/When/Then).



DESIGN

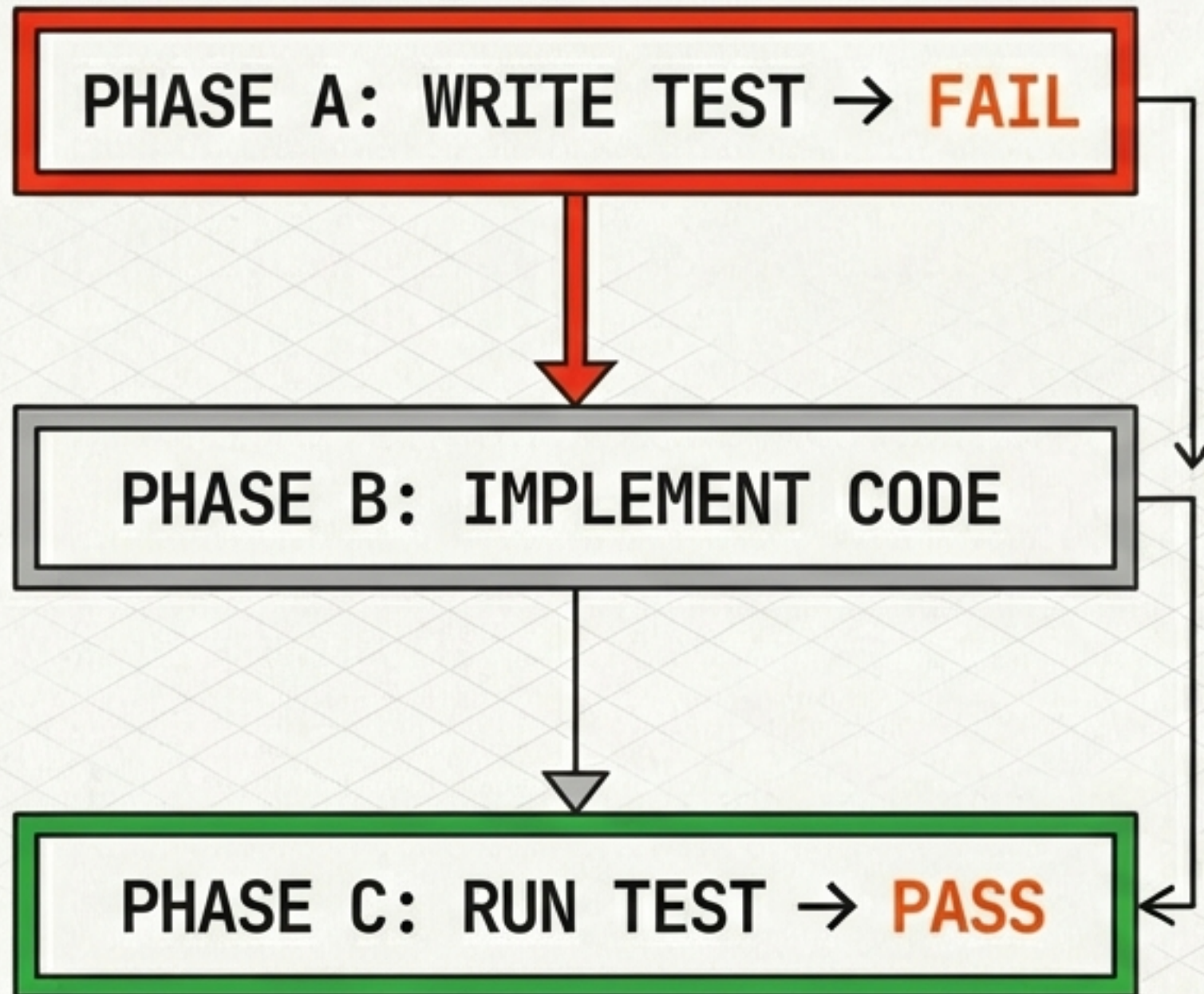
Architectural decisions (e.g., Why POST vs. PUT? Validation logic).



TASKS

Granular checklist of 100+ items (Mocks, Unit Tests, Swagger updates).

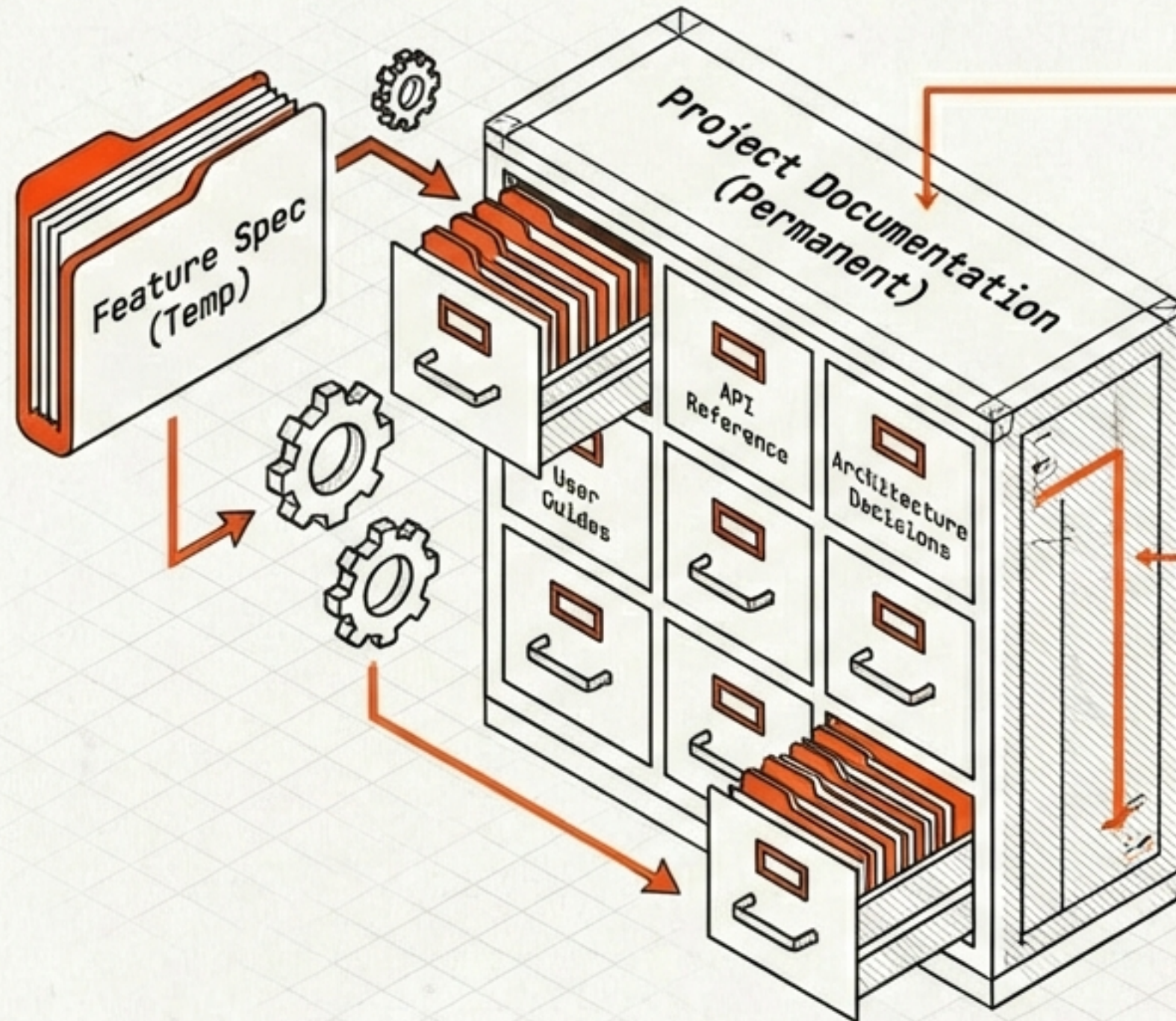
STEP 3: APPLY & EXECUTE



Test-Driven Development (TDD)

The AI writes the test first. It doesn't just say "I'm done"—it provides a log proving the tests passed. A "Verify" command checks consistency between Design and Code.

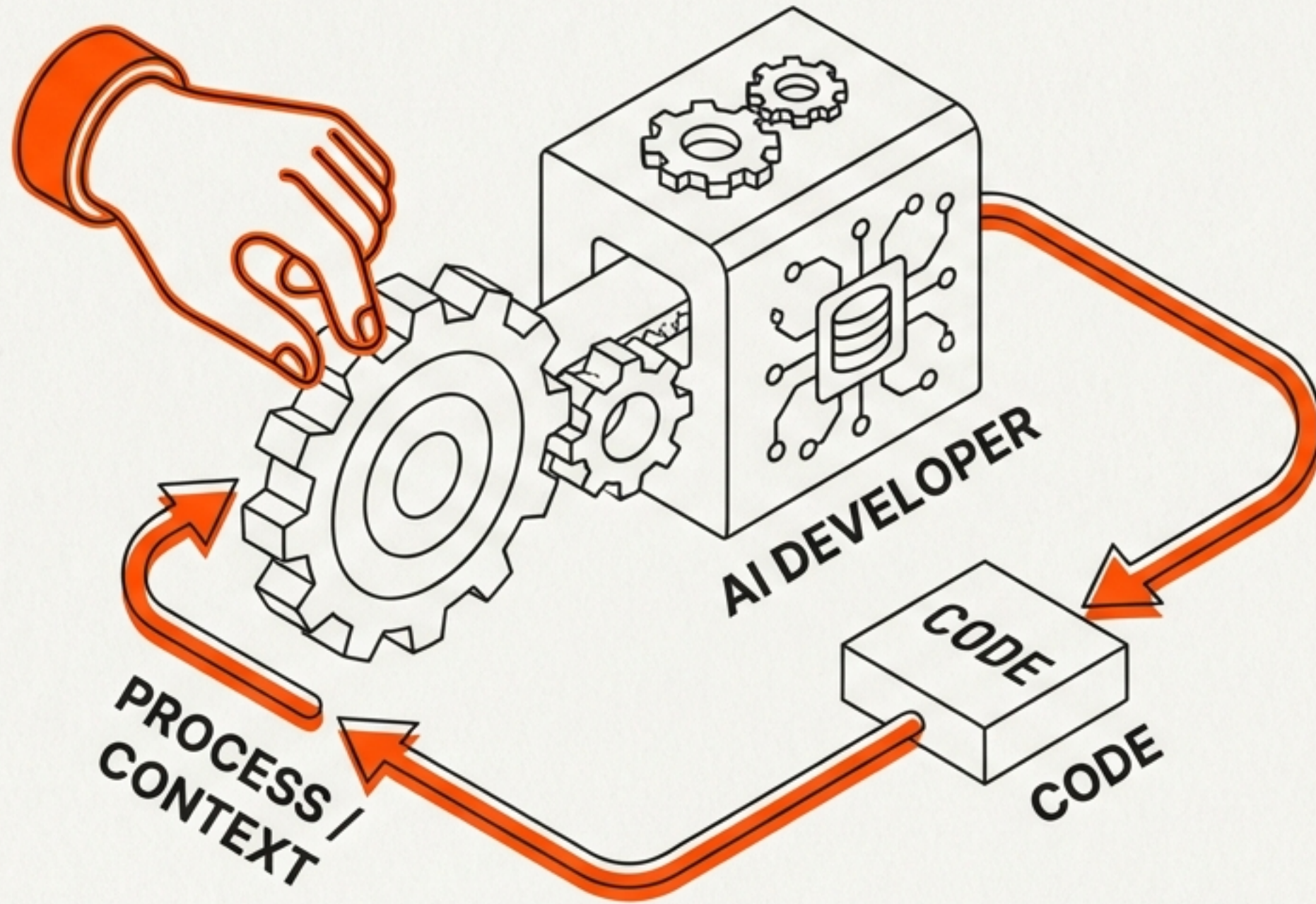
STEP 4: ARCHIVE & THE 'LIVING' DOC



The "Archive" command merges the new feature specs into the permanent project documentation.

Preventing "Confluence Rot":
The repo always contains up-to-date documentation explaining HOW and WHY the system works, readable by both humans and future AI agents.

THE NEW FEEDBACK LOOP



When the AI fails, do not manually fix the code. Fix the Process or the Context.

If a test is missing, update "**Context.md**" to require that test type forever.

You aren't just fixing a bug; you are upgrading the skill level of your AI developer.

THE DEVELOPER SANDWICH

HUMAN: DEFINE & DESIGN →

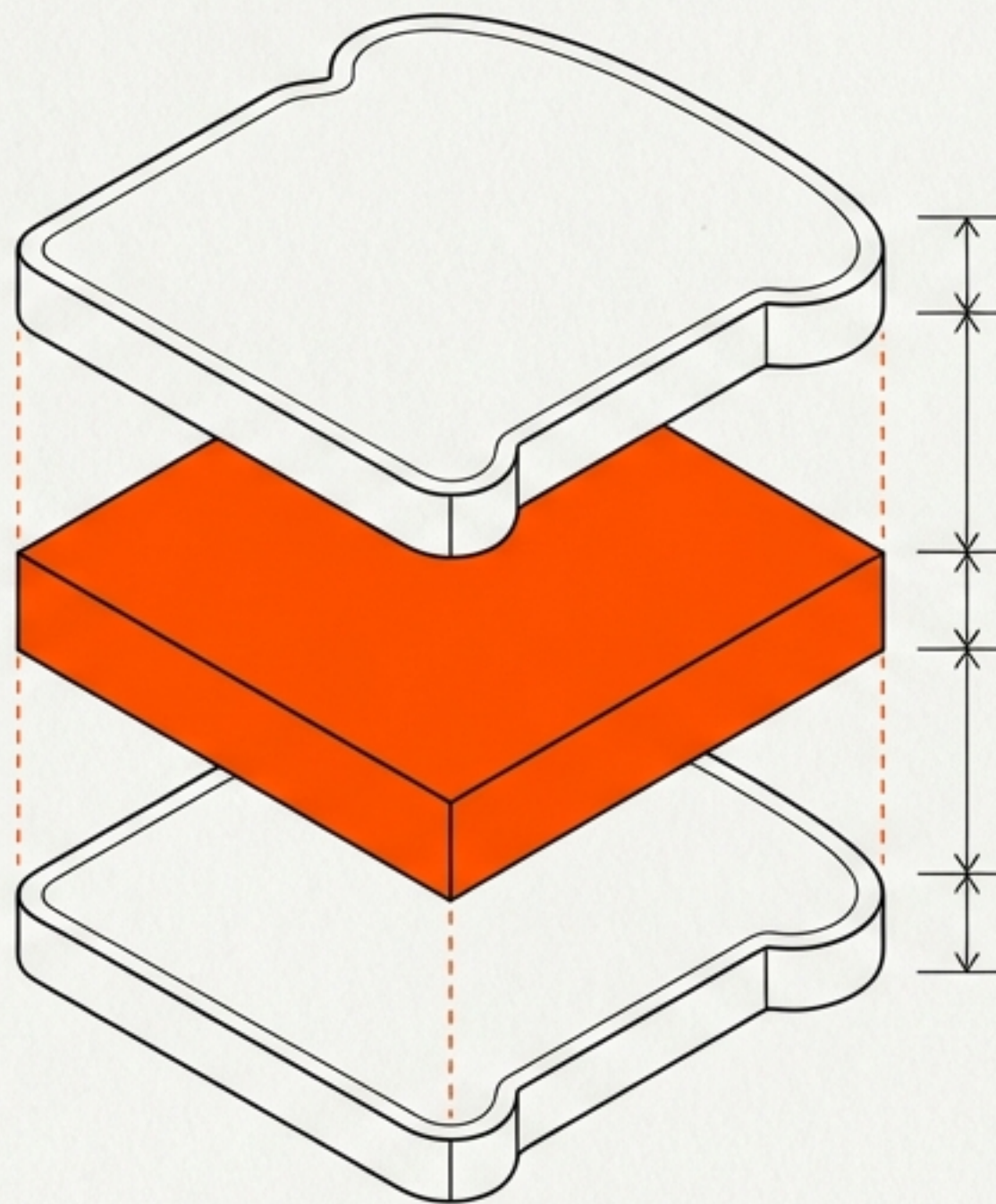
Architecture, Specs,
Contract.

AI: EXECUTE →

Coding, Syntax,
Boilerplate, Unit Tests.

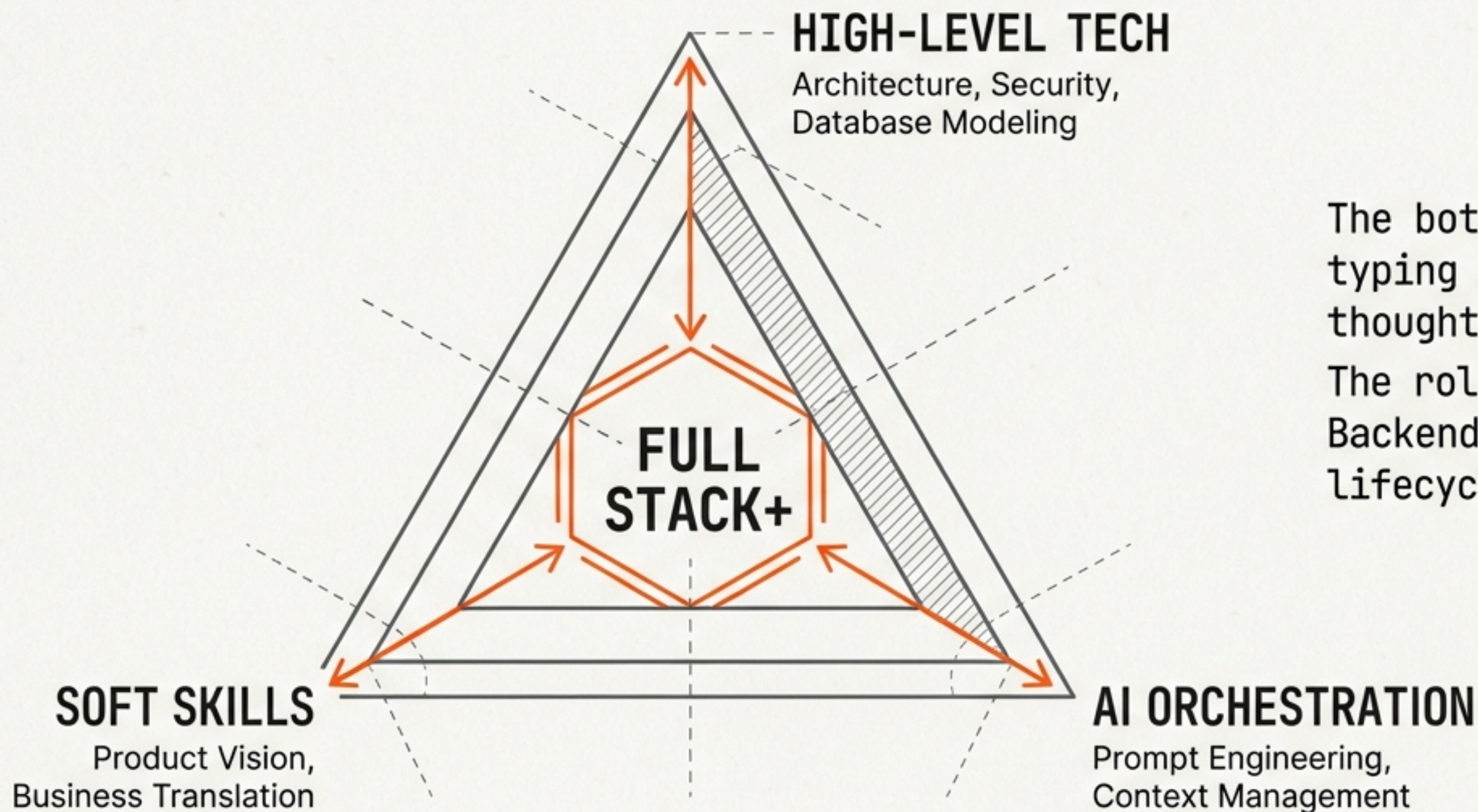
HUMAN: VERIFY & AUDIT →

Review, Deploy, Strategy.



“Our work
shifts to the
edges. The
middle is
the machine.”

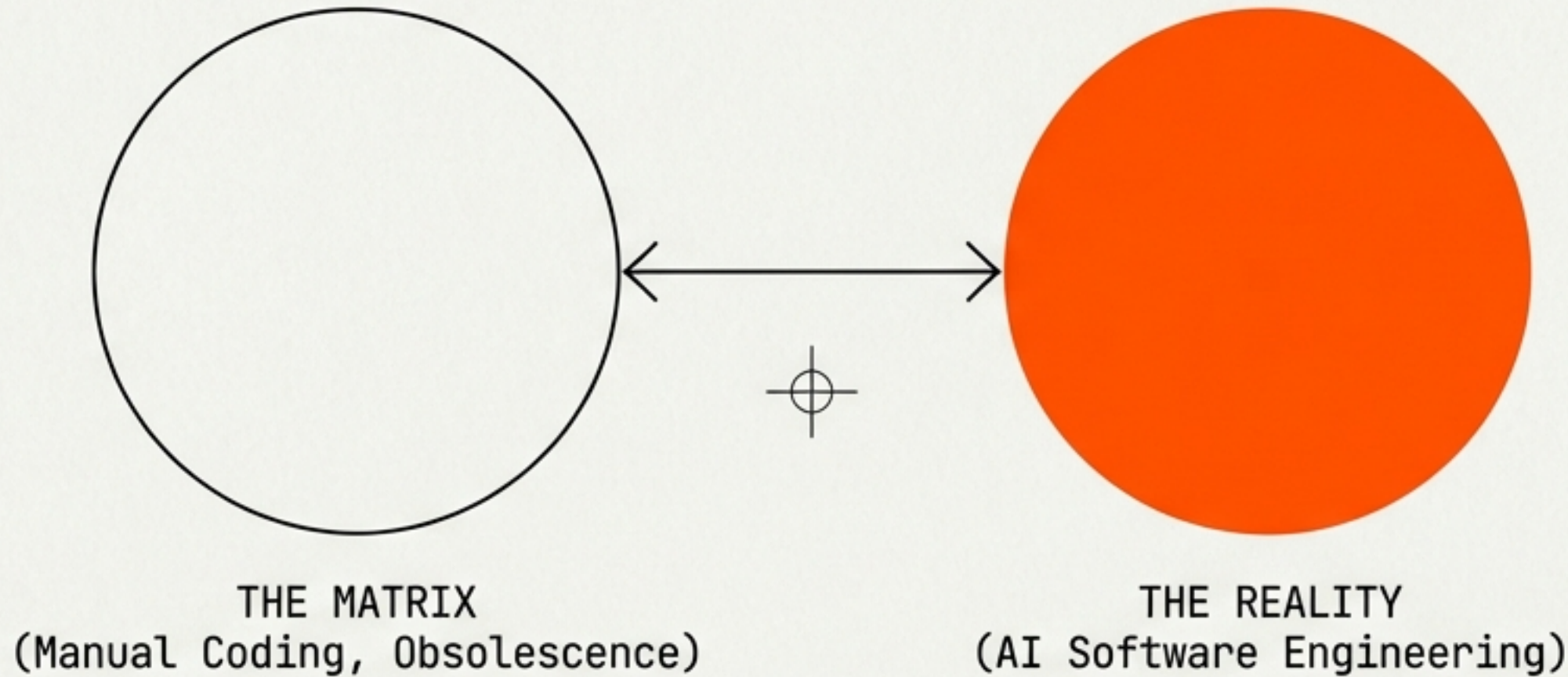
THE RISE OF THE "FULL STACK+" ENGINEER



The bottleneck is no longer typing code. It is clarity of thought.

The role expands from just Backend/Frontend to the full lifecycle of product creation.

THE MORPHEUS MOMENT



“The era of humans writing code has officially ended.” — Ryan Dahl (Creator of Node.js)

The cost of generating code has collapsed to zero. The value of good thinking has skyrocketed.